

SINGLETON Pattern

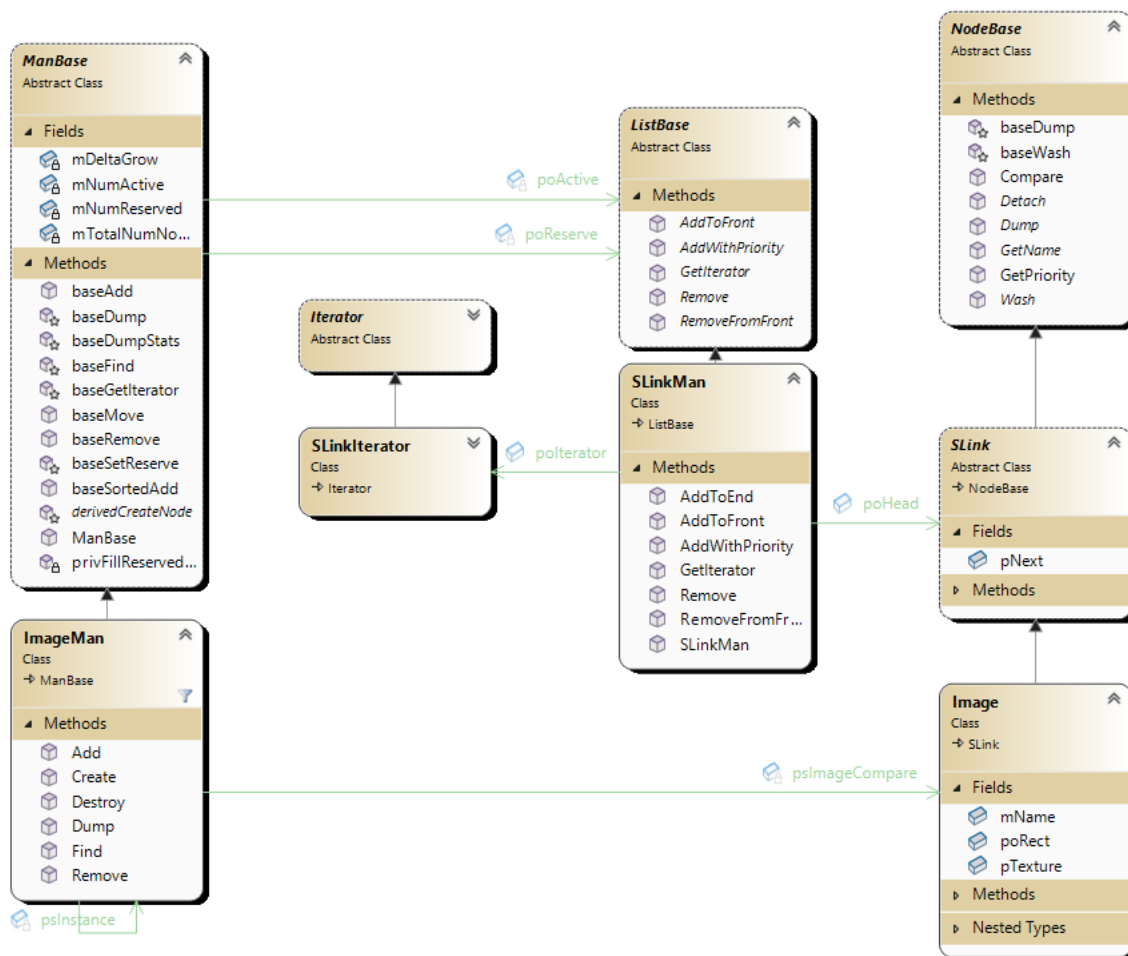
Challenge: Manager can only have one Instance

Problem:

In our Space Invaders implementation, several core systems need to be accessed globally while ensuring only **one instance** exists at a time. For now I will only be focusing on the ImageManager but we used the Singleton pattern for each manager that we created. We needed a way to **restrict instantiation** while allowing global access.

Solution:

To solve this problem, we implemented the **Singleton Pattern**. This ensures that only **one instance** of a class exists throughout the game's lifecycle, and it provides a **global point of access** to that instance.



Pattern description

The **Singleton Pattern** restricts instantiation of a class to a **single instance**, which is globally accessible.

- We implemented this by:
 1. **Making the constructor private** to prevent direct instantiation.
 2. **Creating a static method** to provide controlled access to the instance.
 3. **Storing the instance in a static field**, ensuring it persists throughout the game.
- With the Singleton Pattern, we ensured that key managers like TimerEventManager and SpriteBatchMan were **always initialized once** and shared across the entire game.

Key Object-Oriented Mechanics

The **Singleton Pattern** leverages **encapsulation, global access control, and lazy initialization** to maintain a single instance efficiently.

1. **Encapsulation**
 - The Singleton encapsulates its instantiation logic within a private constructor, ensuring the class controls its own lifecycle.
2. **Global Access Control**
 - A **static method** provides access to the single instance, making it globally available while maintaining controlled initialization.
3. **Lazy Initialization**
 - The Singleton instance is **only created when first accessed**, preventing unnecessary memory usage.

How it is used in Space Invaders....

We applied the **Singleton Pattern** to efficiently manage key game systems, ensuring centralized control and preventing duplicate instances. These systems included:

- **TextureMan** – Managed texture files that contained all game sprites.
- **ImageMan** – Stored all individual sprite images used in the game.
- **SpriteGameMan** – Held the actual sprites.
- **SpriteBatchMan** – Organized multiple sprite batches stored in a linked list.
- **SpriteBoxMan** – Managed all bounding boxes used for collision detection.

- **TimerEventManager** – Controlled all timed events, including alien movement, alien sounds, bomb and UFO spawning, animations, and more.
- **GameObjectNodeMan** – Maintained all tree-based game object composites.
- **ColPairMan** – Managed collision pairs that tested interactions between two tree composites.
- **SoundMan** – Stored all audio files and included an instance of the sound engine.
- **FontMan** – Managed all in-game text elements, such as player scores and high scores.
- **GhostMan** – Tracked deleted game objects for later reuse, optimizing memory management.

By using the **Singleton Pattern** for these managers, we ensured efficient resource handling, consistency across the game.

FLYWEIGHT Pattern

Challenge: Efficiently Managing Large Numbers of Reusable Objects

Problem:

In our Space Invaders implementation, we needed to render a large number of **text characters** for elements such as:

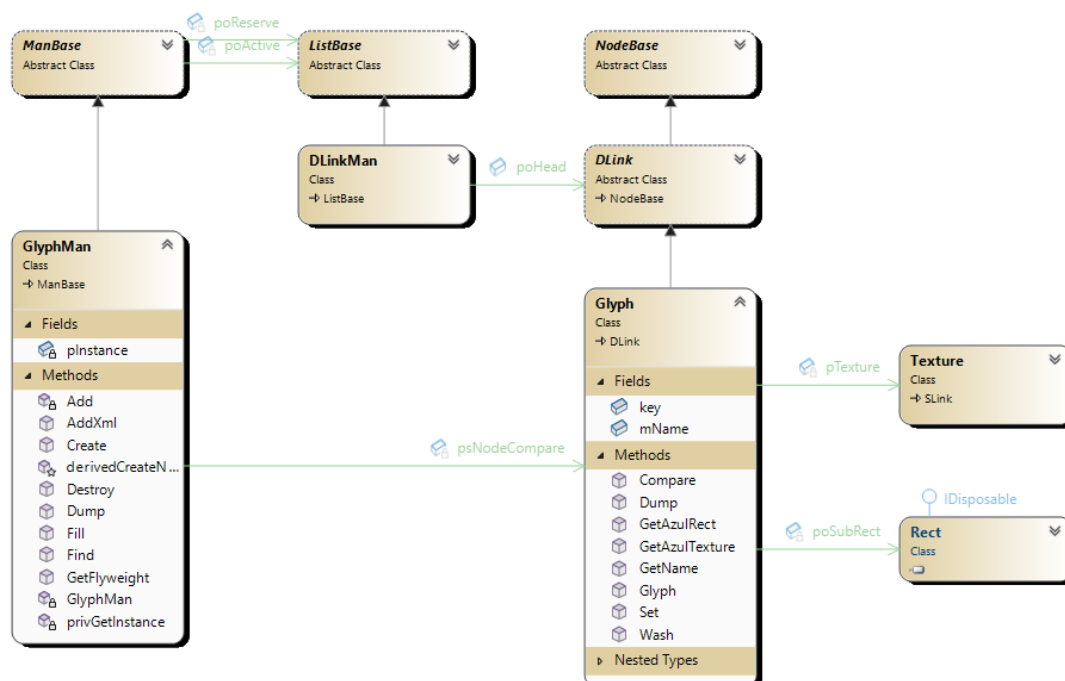
- Player score
- High score
- Game over messages

Each character in the font set was represented as a **sprite**, and rendering individual copies for every occurrence of a letter or number would be complete unnecessary. We needed an efficient way to reuse shared font data while minimizing memory usage.

Solution:

To solve this problem, we implemented the **Flyweight Pattern** in **GlyphMan**, which manages the game's font system.

- Instead of creating a **new sprite for each letter**, we stored a **single shared instance** of each glyph (character sprite).
- When rendering text, we referenced the existing **glyph instances** rather than duplicating image data.



Pattern description

The **Flyweight Pattern** is used to **minimize memory usage** by sharing common data among multiple objects.

- In our case, **each character (glyph) is a flyweight object**, meaning that it is **stored once** and **reused** whenever needed.
- The pattern consists of:
 1. **Intrinsic State (Shared Data):** The actual **image data** for each glyph, stored once in **GlyphMan**.
 2. **Extrinsic State (Unique Data):** The **position and size** of each glyph when rendered on-screen, which varies based on where the text appears.

This separation allows us to **store less data while maintaining full flexibility** in rendering text. One difference that I saw from other Flyweight examples was that when they called `GetFlyweight` if the object did not exist, they created it. In our case we just instantiated all of our Glyphs at the beginning.

Key Object-Oriented Mechanics

The **Flyweight Pattern** relies on **object reuse, separation of intrinsic/extrinsic state, and centralized management to optimize performance**.

1. **Object Reuse**
 - Instead of creating a new object for each letter, we reuse existing glyphs, reducing memory consumption.
2. **Separation of State**
 - **Intrinsic State:** The image data of each glyph (shared).
 - **Extrinsic State:** The text position, scale, and spacing (computed at runtime).
3. **Centralized Management**
 - **GlyphMan** acts as a factory and cache, ensuring that each glyph is only stored once and referenced when needed.

How it is used in Space Invaders....

We applied the **Flyweight Pattern** to **GlyphMan**, optimizing font rendering across the game.

1. **GlyphMan**
 - Loads and stores **one instance per glyph** (letters, numbers, and symbols).
 - Provides access to these glyphs when rendering text.

2. **FontMan**

- Uses GlyphMan to retrieve characters instead of creating new instances.
- Calculates **extrinsic properties** like position, scale, and spacing at runtime.

ITERATOR Pattern

Challenge: Traversing Complex Game Object Hierarchies

Problem:

In our **Space Invaders** implementation, many game objects are structured in a **hierarchical tree** using the **Composite Pattern**. For example:

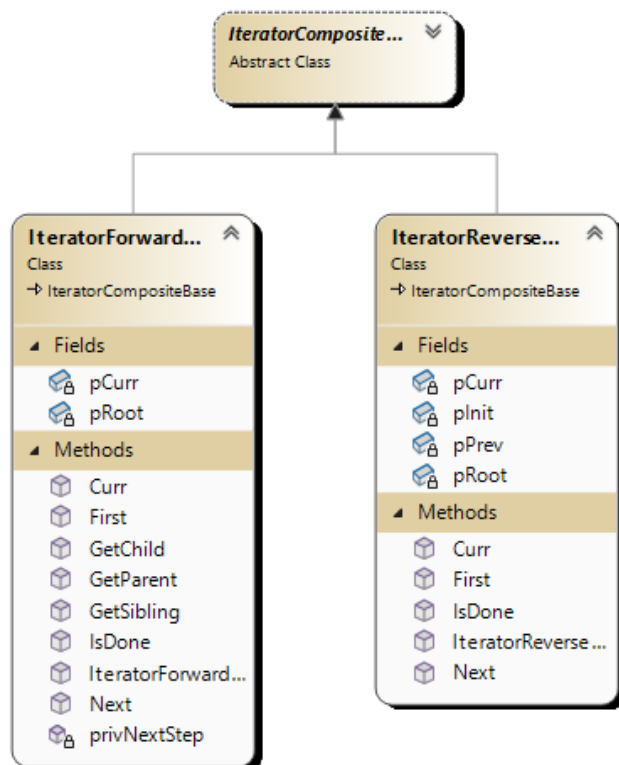
- **AlienGrid** contains **AlienColumns**
- **AlienColumns** contain multiple **Aliens**

To efficiently manage these structures, we needed a way to **traverse and manipulate** all objects without manually handling tree traversal logic. We needed an iterator that could **navigate through the composite hierarchy**, allowing us to **efficiently process all game objects** such as updating, rendering, or deleting them.

Solution:

To solve this, we implemented the **Iterator Pattern**, which enables traversal of hierarchical game objects.

- The **Composite Iterator** allows us to iterate through all children in a composite in a forward and reverse iterator.



Pattern description

The **Composite Iterator Pattern** is an extension of the **Iterator Pattern**, specifically designed to **traverse tree-like structures** in a composite hierarchy.

- **Traditional iterators** only work with **flat data structures**.
- **Composite Iterators** allow us to **traverse nested objects**.
- This approach provides a **consistent and modular way** to process game objects without exposing complex traversal logic.

Key Object-Oriented Mechanics

The **Composite Iterator Pattern** relies on recursion, encapsulation, and polymorphism to traverse hierarchical structures efficiently.

1. Encapsulation

- The iterator encapsulates tree traversal logic, so the game objects do not need to handle traversal manually.

2. Polymorphism

- The iterator works with **GameObject (base class)**, allowing traversal of different object types (**Aliens, Shields, etc.**) without modifying their implementation.

How it is used in Space Invaders....

We applied the **Composite Iterator Pattern** to traverse and process all game objects efficiently.

- 1. AlienGrid (Composite Structure)**
 - **AlienGrid → AlienColumns → Aliens**
- 2. Shields (Composite Structure)**
 - **ShieldRoot → ShieldGrid → ShieldColumns → ShieldBricks**
- 3. Bombs**
 - **BombRoot → Bomb**
- 4. Missiles**
 - **MissileGroup → Missiles**
- 5. Bumpers**
 - **BumperRoot → Bumpers**
- 6. Ships**
 - **ShipRoot → Ship**
- 7. Walls**
 - **WallGroups → Walls**

In addition to all the composite iterators we have regular iterators for every linked list. Most of our singleton managers included double or single linked lists which had their respective iterators that we use.

STRATEGY Pattern

Challenge: Implementing Multiple Bomb Dropping Behaviors

Problem:

In our **Space Invaders** implementation, alien bombs each had their own unique falling animations that we needed to implement. The game included 3 unique falling animations that we needed to implement.

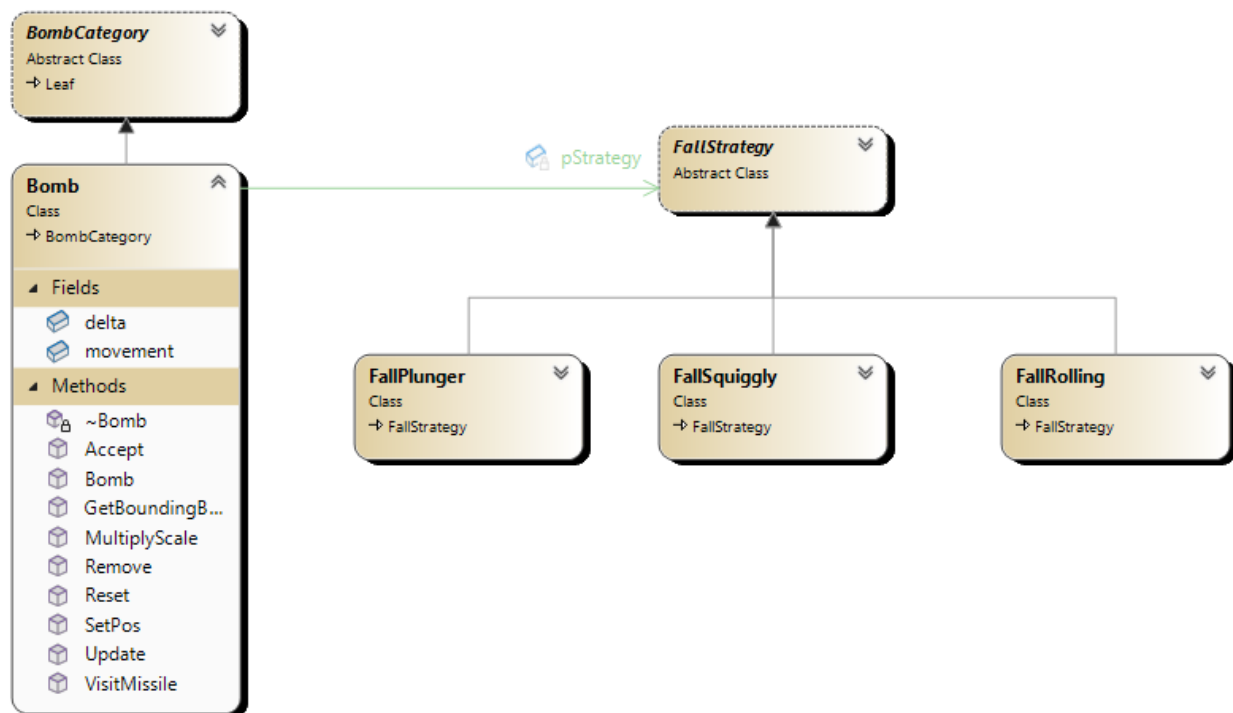
Different bomb-dropping strategies were needed, such as:

- **Plunger Shot** – Moves straight down at a constant speed.
- **Rolling Shot** – Moves down while slightly oscillating side to side.
- **Squiggly Shot** – Moves down in an unpredictable, wavy motion.

Solution:

We implemented the **Strategy Pattern**, which allows different bomb movement behaviors to be encapsulated in separate **strategy classes**.

- Instead of defining movement logic **inside** the Bomb class, we created a **BombMovementStrategy** interface.
- Each bomb type (Plunger, Rolling, Squiggly) **implements a unique movement behavior**.



Pattern description

The **Strategy Pattern** allows an object to dynamically select a behavior from multiple available strategies.

- We defined a **FallStrategy** interface that encapsulates different movement behaviors.
- Concrete strategy classes (**PlungerShot**, **RollingShot**, **SquigglyShot**) implement different movement mechanics.
- The Bomb class **delegates movement** to a strategy instance, allowing easy swapping of behaviors without modifying the base class.

Key Object-Oriented Mechanics

The **Strategy Pattern** leverages **encapsulation, polymorphism, and composition** to create interchangeable behaviors.

1. Encapsulation

- Each movement strategy is encapsulated in its own class, keeping the Bomb class clean and focused.

2. Polymorphism

- The Bomb class interacts with movement behaviors **through a common interface**, making them interchangeable.

How it is used in Space Invaders....

We applied the **Strategy Pattern** to allow bombs have a **different movement behavior** based on its assigned strategy.

FACTORY Pattern

Challenge: Creating Many Different Game Objects

Problem:

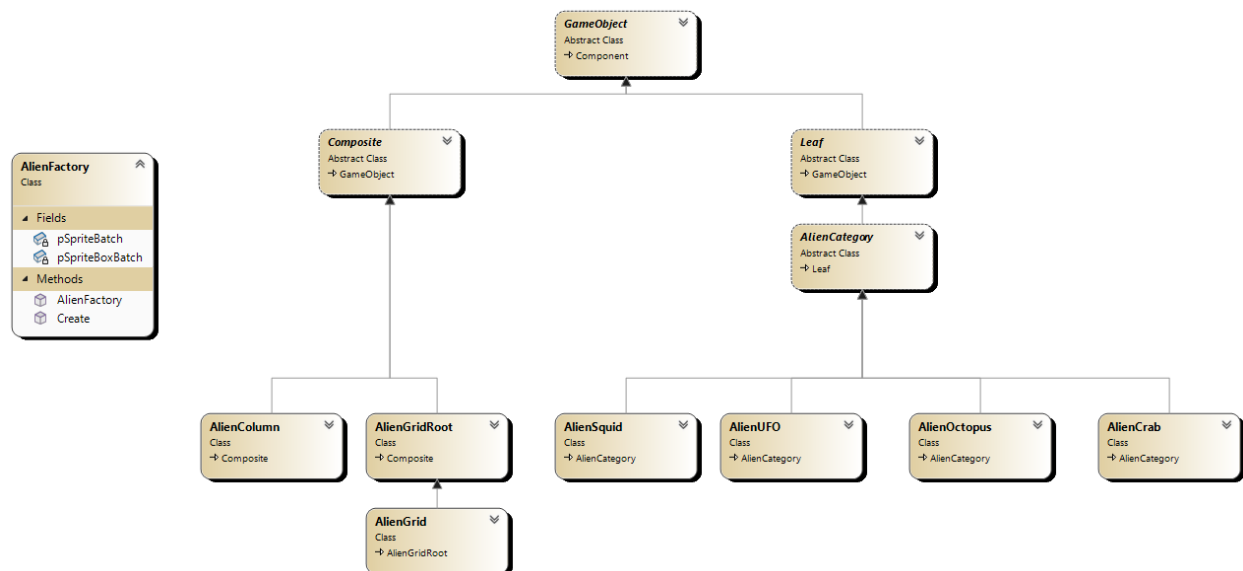
In our Space Invaders implementation, we needed to efficiently create a large number of GameObjects for both the **AlienGrid** and **Shield** structures. Specifically, the **AlienGrid** required:

- **1 AlienGrid** (parent container)
- **11 AlienColumns** (each containing multiple aliens)
- **55 Aliens** (consisting of Squid, Crab, Octopus, and UFO types)

Manually instantiating each of these objects would lead to redundant code and make it harder to maintain or modify the system. We needed a more efficient and scalable solution for object creation.

Solution:

To address this challenge, we implemented the **Factory Pattern**. We created a Create function that accepts an AlienCategory type and optional (x, y) position. This function abstracts the object creation logic, automatically generating the appropriate GameObject based on the given category.



Pattern description

The **Factory Pattern** allows us to create multiple different objects using a single function.

- We implemented an AlienFactory with a Create function, which allowed us to generate:
 - The **AlienGrid**
 - **AlienColumns**
 - Four different **Alien types: Squid, Crab, Octopus, and UFO**
- This approach enabled us to **standardize** GameObject creation. Using a single command, we could instantiate various objects by simply changing one parameter.
- Each created GameObject was then added to its respective composite structure (AlienGrid, AlienColumn, etc.), ensuring a clear and organized hierarchy.

Key Object Oriented mechanics

The **Factory Pattern** leverages three key object-oriented principles to streamline object creation:

1. Encapsulation

- The Factory encapsulates all object creation logic, preventing scattered instantiation code throughout the project.
- This keeps the system modular and simplifies future modifications. For example, I was able to add the UFO later utilizing previously created architecture.

2. Polymorphism

- The Factory Pattern allows us to create different GameObjects using a shared base class.
- Each GameObject (Squid, Crab, Octopus, UFO) inherits from the same parent class, ensuring that they follow a consistent interface while still allowing unique behaviors.

3. Abstraction

- The client code does not need to know how objects are instantiated. For this project this didn't matter but is very useful in other applications.

How it is used in Space Invaders....

We applied the Factory Pattern to **both Aliens and Shield bricks** to streamline object creation.

1. Aliens:

- First, an AlienGrid was created to contain all the aliens.

- Each AlienColumn was then instantiated inside the grid.
- Each column contained **5 Aliens**, following a predefined pattern:
 - **1 Squid** (top)
 - **2 Crabs** (middle)
 - **2 Octopi** (bottom)

2. Shields:

- We created **4 Shield Grids**, each composed of **6 Columns**.
- Each column contained **10 Shield Bricks**, some with different sprites attached.
- The Factory allowed us to create all these objects efficiently by modifying just one parameter in the Create function.

Without the **Factory Pattern**, we would have had to manually instantiate each GameObject with individual constructors, leading to redundant code and a system that would be difficult to modify. Instead, the Factory allowed us to **simplify, standardize, and optimize** object creation.

PROXY Pattern

Challenge: Optimizing Repeated Sprite Usage

Problem:

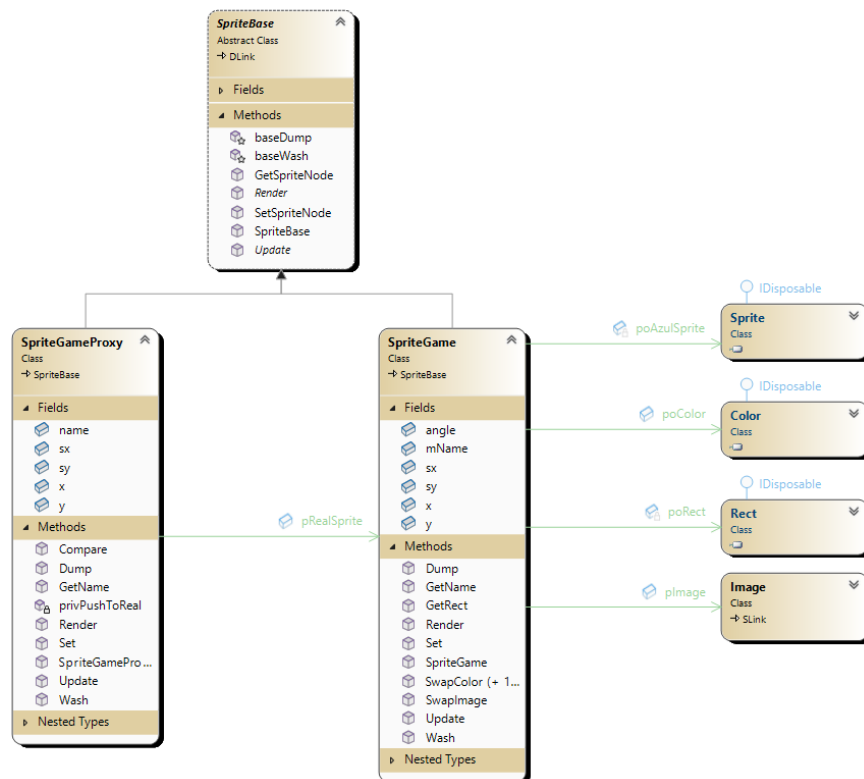
In our **Space Invaders** project, we had a large number of sprites that shared the same appearance but had different **positions and transformations**. Creating a **separate instance** of each sprite with all its associated data (textures, colors, collision boxes) would be **inefficient in terms of memory and performance**.

Additionally, we needed a way to **efficiently update** the position and scale of these sprites without duplicating unnecessary data.

Solution:

To solve this, we implemented the **Proxy Pattern**, which allowed us to create **lightweight proxy objects** that reference a **single, shared RealSprite**. The **GameSpriteProxy** acts as a placeholder, storing only essential attributes such as **position (x, y)** and **scale (sx, sy)**, and delegates rendering and updates to the **RealSprite** when needed.

This approach enabled us to **reduce memory usage**, **simplify object management**, and **improve performance** while still maintaining normal sprite behavior.



The **Proxy Pattern** provides a lightweight interface to a heavier object, allowing for **efficient interaction** without directly managing the full object's complexity.

At the core of this pattern are two key components:

1. **Proxy Object (GameSpriteProxy)**
 - Stores **only the essential data** (name, position, scale).
 - Maintains a **pointer to the RealSprite**.
 - Acts as an interface for interacting with the RealSprite.
2. **Real Object (GameSprite / RealSprite)**
 - Contains the **full sprite data** (textures, images, colors, animations).
 - Handles the actual rendering and logic processing.
 - Is shared among multiple proxies to **reduce memory usage**.

The **GameSpriteProxy** acts as an intermediary, allowing multiple proxy instances to reference a **single RealSprite**.

Key Object-Oriented mechanics

Key Object-Oriented Mechanics

1. **Encapsulation**
 - The **proxy hides the complexity** of the RealSprite and exposes only the necessary attributes, making object interaction cleaner and more manageable.
2. **Polymorphism**
 - Both **GameSpriteProxy** and **GameSprite** inherit from **SpriteBase**, which defines common methods such as Update() and Render().
 - This ensures that the **proxy can be used interchangeably** with the real object in the game engine.
3. **Delegation**
 - The **proxy forwards calls** to the RealSprite after updating its essential attributes.
 - This allows updates to be handled efficiently while **keeping the real object's state synchronized**.
4. **Memory Efficiency**
 - Since **multiple proxies can point to a single RealSprite**, we avoid redundant data storage.
 - Instead of creating **55 separate alien sprites**, we create **only 3 RealSprites (Squid, Crab, Octopus)** and reference them through proxies.

How it is used in Space Invaders....

We implemented the Proxy Pattern in several areas of our **Space Invaders** game:

1. Alien Sprites Optimization

- Instead of creating **22 separate Crab sprites**, we created **22 Crab proxies** that all pointed to a **single Crab RealSprite**.

- This significantly **reduced memory consumption** while ensuring that all crabs could be individually positioned and animated **simultaneously**.

2. Shared Alien Animation

- Because all proxy objects **referenced the same RealSprite**, updating the **RealSprite's texture or animation** automatically **updated all proxies** as well.
- This meant we could animate **55 aliens using just 3 RealSprites**, making animations highly **efficient and scalable**.

3. Shield Bricks Optimization

- The shields consisted of **108 identical bricks** spread across four shields.
- Instead of creating **108 separate RealSprites**, we used proxies that referenced a **small set of shared RealSprites**, dramatically reducing rendering overhead.

STATE Pattern

Challenge: Managing Player Ship States and Movement Constraints

Problem:

In **Space Invaders**, we had to impose certain **constraints** on the player ship's behavior based on its current state. The ship had to exhibit **different behaviors** under different conditions, including:

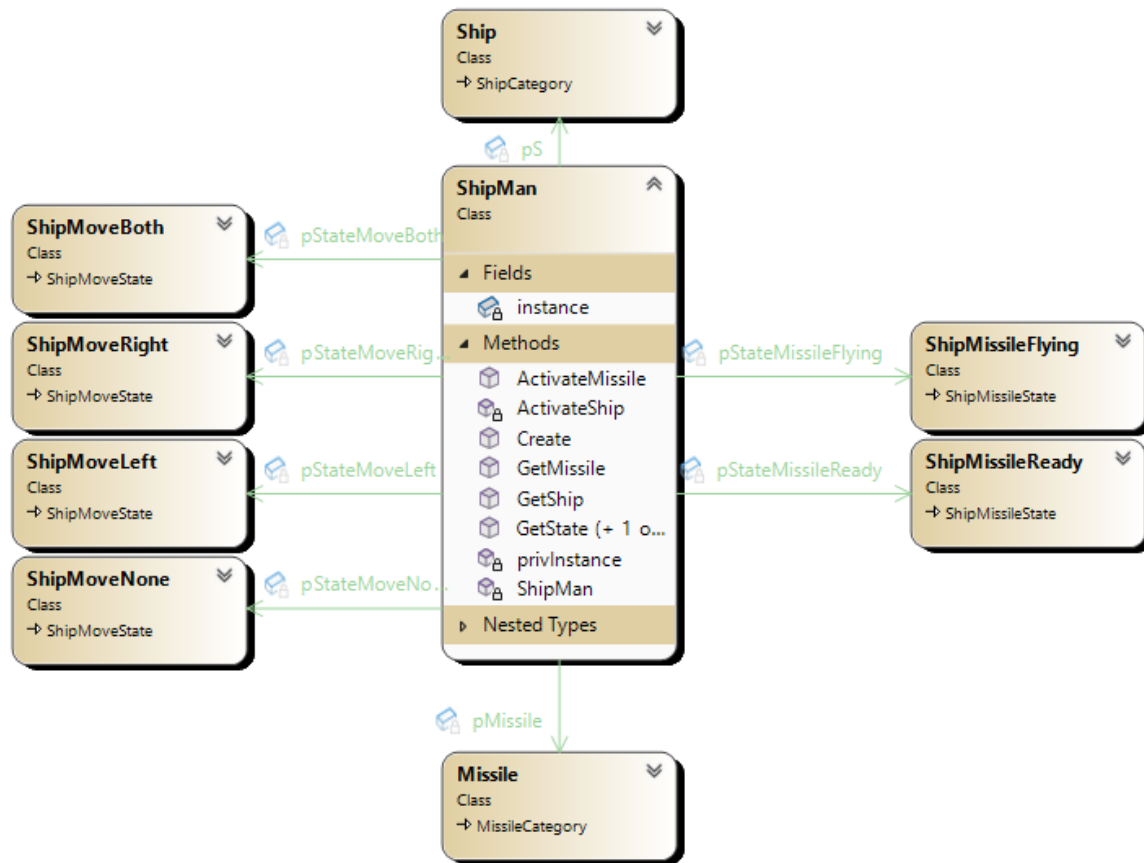
1. **Movement Constraints** – The player could only move left or right until they hit the respective left and right bumpers of the screen.
2. **Missile Limitation** – The player could shoot only **one missile at a time**, with the missile being removed from the screen after a shot or if it collided with an alien or an obstacle.
3. **Inability to Move When Hit by a Bomb** – If the player was hit by an alien bomb, they should **freeze in place**, preventing any movement until the ship respawned.

We needed a solution that could **handle different player states** and **encapsulate behavior changes** based on those states.

Solution:

To address this, we applied the **State Pattern**, which allows us to **manage and change the player ship's behavior dynamically** based on its current state. By using the **State Pattern**, we encapsulated the ship's behaviors into distinct state objects, each representing a particular state of the ship's behavior.

- The **State Pattern** helped us to **modularize** the different behaviors (e.g., movement, missile firing, freezing when hit by a bomb) into **separate state classes**.
- The **ShipMan** maintains a **current state** and delegates the logic of each behavior to the corresponding state object. This makes it **easy to switch between states** without cluttering the ship's logic with numerous conditionals.



Pattern description

The **State Pattern** allows an object to change its behavior when its **internal state** changes. The object will appear to change its class, as it will behave differently depending on its current state.

- In our implementation, the **Ship** class uses a **ShipMan** to define behaviors for different states (e.g., moving, shooting).
- **Concrete states** (e.g., **ShipMoveRight**, **ShipMoveBoth**, **ShipMoveNone**) implement specific behaviors.
- The **ShipMan** interacts with all the different states and delegates all behavior-specific actions (like moving, shooting) to the currently active state.

This pattern makes the player ship's behavior **flexible**, **maintainable**, and **easy to extend**.

Key Object-Oriented Mechanics

The **State Pattern** leverages **polymorphism** and **composition** to manage different states and encapsulate behavior changes.

1. Polymorphism

- Each state implements the **State interface**, ensuring that the ship can change behavior by switching between different states. The ship interacts with its current state, and the actual behavior is determined by the state class.

2. Encapsulation

- The logic for each specific behavior (movement, missile firing, freezing) is encapsulated within its corresponding state class.

How it is used in Space Invaders....

We applied the **State Pattern** to manage the player ship's behavior, ensuring that the ship could only perform certain actions under the correct conditions. We had two bumpers one on the left and one on the right. When the Ship collides with either of the bumpers, we fire off observers which change the state of the ship to ensure that the ship will no longer be able to move in a restricted direction. We also had a missile observer that would notify us whenever a missile hit something whether it was an Alien, a UFO, a Bomb, or the top of the screen. Once the missile collided with something we would fire an observer switching the ship missile state from flying to ready to indicate that a new missile could be fired. The **State Pattern** allows us to **modularize** and **encapsulate player behaviors** based on the player's state. Without it, we would have to use **complex conditional logic** for each player action (e.g., movement, shooting, freezing), which would make the code difficult to maintain and extend.

VISITOR Pattern

Challenge: Efficiently Handling Collision Detection

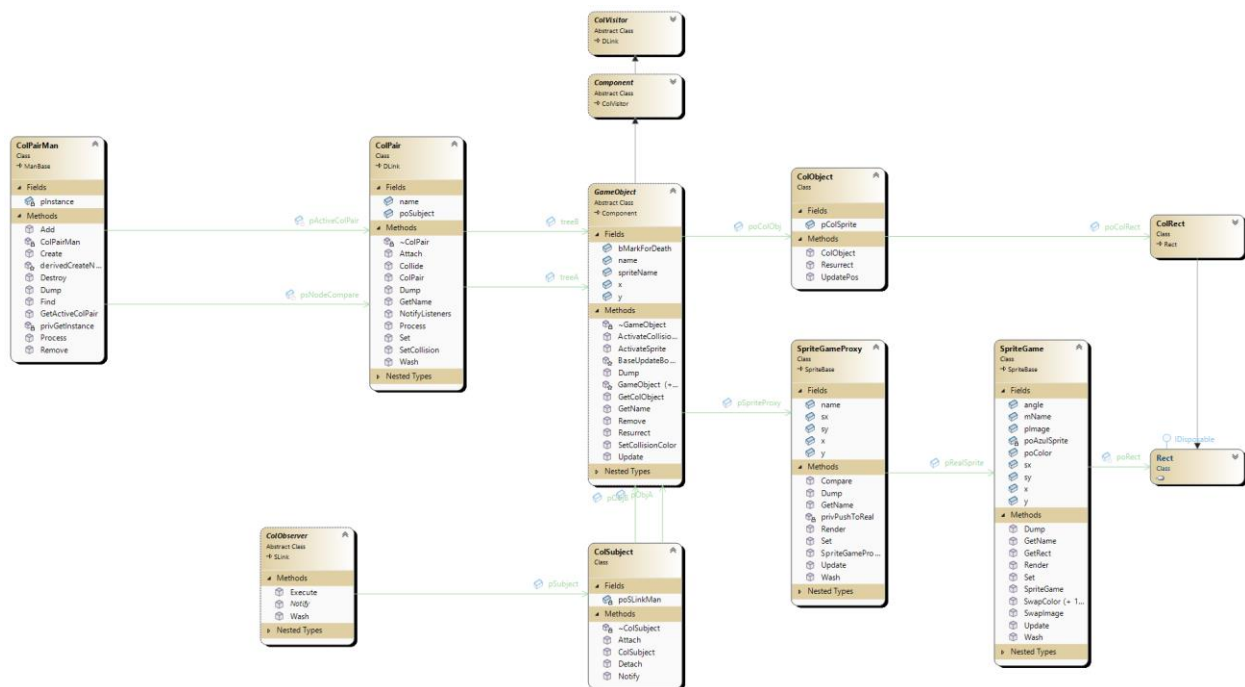
Problem:

In our **Space Invaders** project, we needed to handle complex **collision detection** logic between various game objects. The collision detection had to be **efficiently applied to multiple combinations** of objects that were organized in **composite structures** (i.e., hierarchical trees). The collision logic needed to be **extendable, modular**, and able to handle different object types.

Solution:

To address this, we implemented the **Visitor Pattern**, which allows us to **separate the collision logic** from the game objects themselves. This pattern is ideal for operations that involve multiple **object types** and need to be **extended** over time.

Using the **Visitor Pattern**, we created a **CollisionVisitor** that can "**visit**" the various **composite objects** (e.g., AlienGrid, ShieldRoot, Missile, Bomb, PlayerShip, etc.). Each object will **accept** the visitor and delegate the appropriate collision checks based on its type. This approach ensures that collision logic is **decoupled** from the game objects, making it easier to maintain and extend.



Pattern description

The **Visitor Pattern** is a design pattern that allows you to separate an algorithm (in this case, collision detection) from the object structure (composites and game objects). The key idea is to

define a **visitor** class that **visits** different elements of the structure and performs operations on them.

How the Visitor Pattern Works in Our Project:

1. **CollisionPairManager** holds a list of **collision pairs**, where each pair consists of two composite trees that need to be tested for collision (e.g., **AlienGrid vs Left Wall**, **Missile vs AlienGrid**).
2. Each game object (e.g., **AlienGrid**, **Missile**, **Bomb**) implements the **accept** method, which **accepts** a visitor and passes itself to the visitor to perform the collision detection.
3. The **CollisionVisitor** class defines various virtual **visit methods** for handling different combinations of collisions that we can choose to override in each individual **GameObject** based on necessity.
4. The visitor methods are **polymorphic**, meaning they can handle different combinations of objects (such as **Missile vs AlienGrid**, or **Bomb vs PlayerShip**) without modifying the game object classes.

Key Object-Oriented Mechanics

The **Visitor Pattern** utilizes **polymorphism** and **composition** to allow **dynamic operation dispatching** based on the type of the object being visited.

1. Polymorphism

- The **CollisionVisitor** class has **different visit methods** for each combination of objects. For example, it has methods for handling **AlienGrid vs Missile** collisions and **Missile vs UFO** collisions. Each of these methods is called based on the actual types of the objects involved in the collision.

2. Composition

- The game objects are **composite structures** (e.g., **AlienGrid** contains **AlienColumns**, and each **AlienColumn** contains multiple **Aliens**). The **accept** method of each game object accepts a **visitor** and delegates the collision logic to the appropriate visitor method.

How it is used in Space Invaders....

In our implementation, the **Visitor Pattern** allowed us to **modularize the collision detection logic** for the various combinations of game objects. Here's how we implemented the pattern:

1. CollisionPairManager

- This class manages the **collision pairs**. Each **collision pair** consists of two objects (composites or game objects) that need to be tested for collision. The manager iterates over the pairs and checks if their boxes are colliding.

2. CollisionVisitor

- The CollisionVisitor class contains virtual visit methods for all types of game objects that can collide. We override these methods inside of the GameObject so that we only check collisions that we care about.

3. Game Object Classes

- Each game object implements the **accept** method. The method delegates the responsibility of handling collisions to the appropriate **visit** method.

4. All Tested Collision

- AlienGrid vs left and right bounding walls
- AlienGrid vs ShieldRoot
- Missile vs AlienGrid
- Missile vs UFO Grid
- Missile vs top wall
- Missile vs Shield
- Bumper vs PlayerShip
- Bomb vs Missile
- Bomb vs bottom of the screen
- Bomb vs Shield
- Bomb vs PlayerShip

ADAPTOR Pattern

Challenge: Integrating with the Azul Game Engine

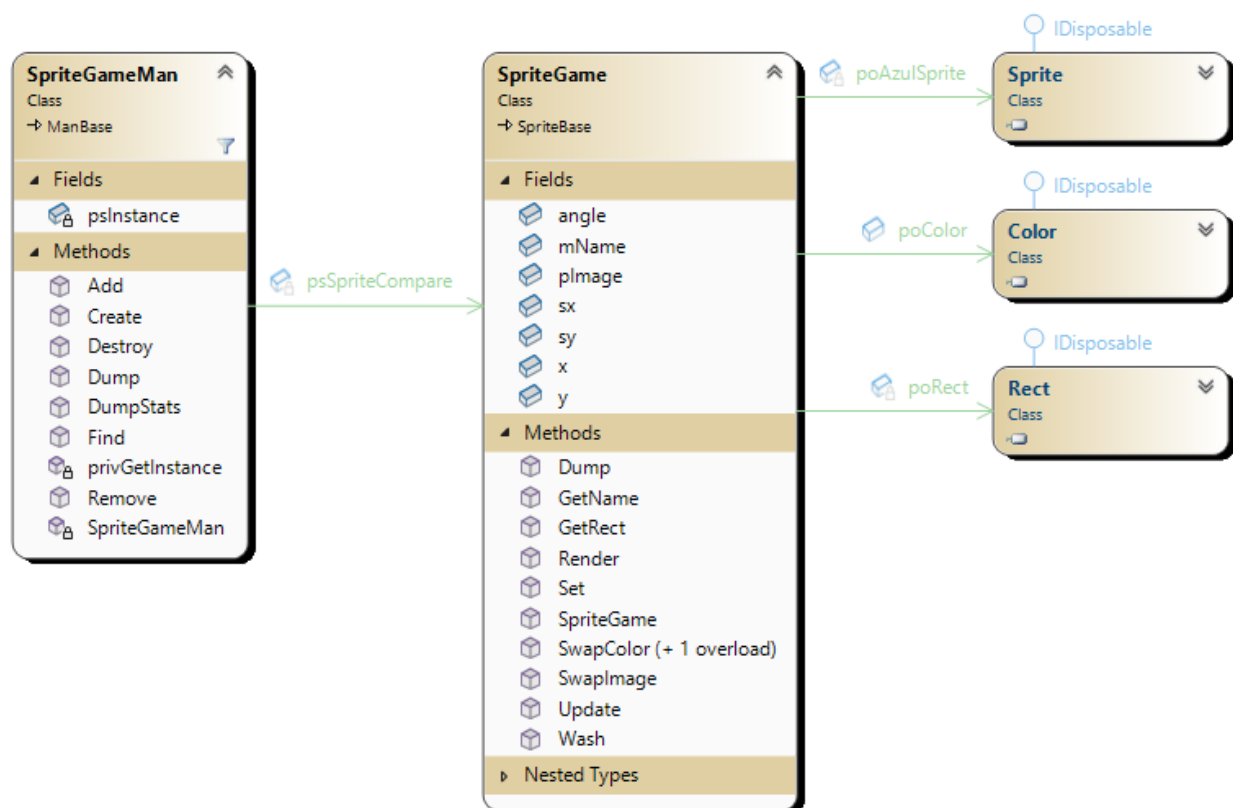
Problem:

In our **Space Invaders** project, we used a custom game engine called **Azul**, which provided several core components for game development, including **Sprite**, **Color**, and **Rect** objects.

The key challenge was to **adapt the Azul engine's objects** into a format that we could use seamlessly. The goal was to allow our **SpriteGame** object to easily interact with the Azul components.

Solution:

We applied the **Adaptor Pattern** to solve this problem. The **Adaptor Pattern** allowed us to interact with the **SpriteGame** object instead of the Azul engines object ourselves.



Pattern description

The **Adaptor Pattern** is a structural design pattern that allows objects with incompatible interfaces to work together. It acts as a **wrapper** or **bridge** between two incompatible interfaces, making them compatible by **converting one interface into another**.

In our case:

- The **SpriteGame** object allows us to indirectly interact with the **Azul Sprite**
- The **Adapter Pattern** allowed us to **wrap** the **Azul Sprite** in an adapter class that provided the extra functionality needed by **SpriteGame**.

Key Object-Oriented Mechanics

The **Adapter Pattern** utilizes **composition** and **encapsulation** when adapting the interface of an object. Key object-oriented principles involved in the implementation:

1. Composition

- The **Adapter class** holds a reference to the original object (in our case, the **Azul Sprite**) and acts as a **wrapper** around it. This allows the Adapter to **add new behavior** to the object (such as adding **x, y, scaleX, scaleY**).

2. Encapsulation

- The **Adapter Pattern** encapsulates the logic of dealing with different interfaces. The client code (i.e., **SpriteGame**) interacts with the adapter rather than directly interacting with the **Azul Sprite** object, thus hiding the complexity of the Azul components.

How it is used in Space Invaders....

In our implementation, we used the **Adapter Pattern** to bridge the gap between **SpriteGame** and **Azul's Sprites, Color and Rect**.

OBSERVER Pattern

Challenge: Managing Collision Notifications and Updates

Problem:

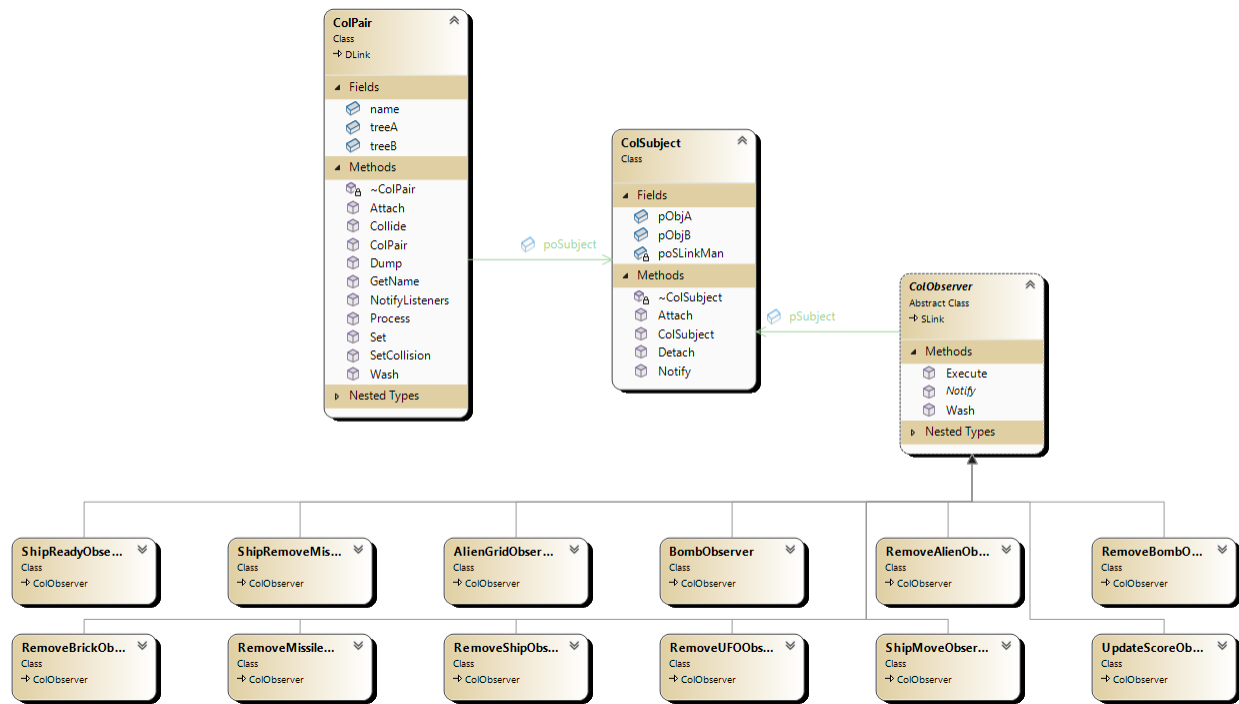
In our **Space Invaders** project, we needed a way to **notify game objects** about collisions and other game events (such as **alien movement**, **missile firing**, and **player interactions**) in a **centralized** and **efficient** manner. We had a variety of collision scenarios and objects that needed to be updated when certain events occurred. These objects included **Aliens**, **Missiles**, **Shields**, **Bombs**, and others, and the collisions involved were numerous and complex.

To make this system scalable, flexible, and decoupled, we needed a way to manage the **notification** of these events without hard-coding the logic for each object directly. Specifically, we wanted the game objects to **observe** certain events (such as collisions) and react accordingly.

Solution:

To handle this, we utilized the **Observer Pattern**. The **Observer Pattern** allowed us to define a **centralized event mechanism** for **collision notifications**. This pattern decoupled the event generation (collision detection) from the event handling (object updates), making the system more flexible, maintainable, and extensible.

In the **Observer Pattern**, we defined **observers** that **watched** specific events (such as **collision pairs**) and performed the necessary updates when those events were triggered.



Pattern description

The **Observer Pattern** is a behavioral design pattern where an **object** (the **subject**) maintains a list of **observers** that are notified when an event occurs. This pattern is used when one object (the subject) needs to notify other objects (observers) about a change without knowing exactly who or what those objects are.

- The **ColSubject** holds a list of **Observers** (game objects like **Aliens**, **Missiles**, **Shields**, etc.) and notifies them when a specific event occurs.
- The **Observers** are registered with the **Subject** and update themselves when the **Subject** changes or an event is triggered.
- The **Observers** implement an interface called **ColObserver** which holds an **update** method, allowing subjects to be notified and handle events independently.

The **Observer Pattern** is especially useful when the system needs to notify multiple objects about an event, and the objects should be able to react independently without tightly coupling the event logic to the objects.

Key Object-Oriented Mechanics

The **Observer Pattern** is a powerful and flexible way to handle notifications and event-driven behavior. In our implementation, the pattern used the following key object-oriented principles:

1. Loose Coupling (Decoupling)

- The **Observer Pattern** allows for **loose coupling** between the **subject** and the **Col observers**. The subject does not need to know the details of its observers. It simply notifies them when an event occurs. The observers, in turn, handle the event appropriately.

2. Polymorphism

- The **Observers** implement a **common interface** (**ColObserver**) that allows the **subject** to treat all observers in the same way. When an event occurs, the **subject** calls the **update()** method on all registered observers, relying on polymorphism to handle different observers' specific behavior.

3. Composition

- The **Observer Pattern** relies on **composition** rather than inheritance. Observers are **composed** of their own behavior and are added to the **subject** dynamically. This allows the system to **easily extend** the number of observers without modifying the subject itself.

How it is used in Space Invaders....

In our **Space Invaders** project, the **Observer Pattern** was applied to **collision detection**. Here's how the system worked:

1. **ColPairManager**

- We used a **collisionPairMan** that **tracked** and **detected collisions** between different game objects, such as **Missiles**, **Aliens**, **Bombs**, and **Shields**. This manager had a **ColSubject** which kept track of **all registered observers**.

2. **Observers (Game Objects)**

- Game objects such as **Aliens**, **Shields**, **Missiles**, and **PlayerShip** acted as the **observers**. They were **notified** by the **collision manager** when relevant collisions occurred (e.g., when a **Missile** collided with an **Alien** or a **Bomb** hit the **PlayerShip**).
- When a collision was detected, the **Observer** (e.g., **Missile**) would respond accordingly (e.g., removing the **Missile** from the screen and increasing the player's score).

3. **Registration of Observers**

- Observers (game objects) were **registered** with the **ColPairMan** during instantiation. Whenever a collision event was detected, the **ColPairMan** would notify all relevant observers using the **update()** method.

4. **Collision Handling**

- **AlienGridObserver** was used to notify when the AlienGrid hit the left or right wall
- **RemoveAlienObserver** was used to notify when a Missile hit an Alien.
 - It would calculate how many aliens were on the grid and if there were no more then it would notify the game that the player had completed a level. The level would the restart at a higher difficulty.
 - It would also play a sound when the Alien was shot, mark it for death add the AlienExplosionSprite in its place. Then fire a command with a 0.2 second delay to remove the explosion sprite.
- **RemoveBombObserver** was used to notify when a Bomb hit a missile or a bomb hit the bottom of the screen
 - The bomb observer would mark the bomb for death add the AlienShotExplosion in its place. Then fire a command with a 0.2 second delay to remove the explosion sprite.
- **RemoveBrickObserver** was used to notify when a missile, a bomb, or an alien collision occurred marking the brick for death.

- **RemoveMissileObserver** would remove the missile from the screen when it collides with an Alien, Wall, or UFO by marking it for death.
- **RemoveShipObserver** would notify when a bomb hit the users ship.
 - First it would access a bool in the ship to check if the ship is currently dying. Ensuring that the ship can only die once
 - Then it finds the bombRoot and iterates through all of the current bombs setting their movement to 0 stopping the bombs midair to play the death animation.
 - Then it calls the soundMan to play an explosion sound.
 - Next it calls TimerEventManager.PauseForDeathUpdate(4f) This function loops through the entire TimerEventManager and adds 4 seconds to every event except the animation event in order to let the death animation play.
 - Then we add two more commands. The first plays the death animation. The second waits for 4 seconds and then resets the players ship and removes one life. I go into further detail in the Command Doc.
- **RemoveUFOObserver** acts similar to the **RemoveAlienObserver**.
 - It would also play a sound when the UFO was shot, mark it for death add the SaucerExplosion in its place. Then fire a command with a 0.2 second delay to remove the explosion sprite.
 - Next however it would add a command to the EventMan that would display the UFO's current value. I go into further detail in the Command Doc.
- **ShipMoveObserver** would change the ShipState depending on which bumper it hit.
 - If it hit the left bumper it would switch states to only allow the ship to move right and vice versa.
- **ShipReadyObserver** would notify the user when the missile collided with something thus destroying the missile.
 - The observer would switch the Ships missile state from flying to ready.
- **ShipRemoveMissileObserver** would notify us when the missile hit the top of the screen.
 - We placed an PlayerShotExplosion sprite in its location and removed it after 0.2 seconds using a command.
- **UpdateScoreObserver** would be notified whenever an Alien or UFO was shot

- First the observer would get the current score stored as a string in the FontMan. The string is then converted to an int.
- Because the UFO has a variable value based on the number of shots the player has taken, the observer would check the ships current shot count and enter a switch statement to determine the value.
- Depending on which Alien it hit the score would increment.
- The score would then be formatted back into a string and updated.

COMMAND Pattern

Challenge: Managing Multiple Actions in a Timed Order

Problem:

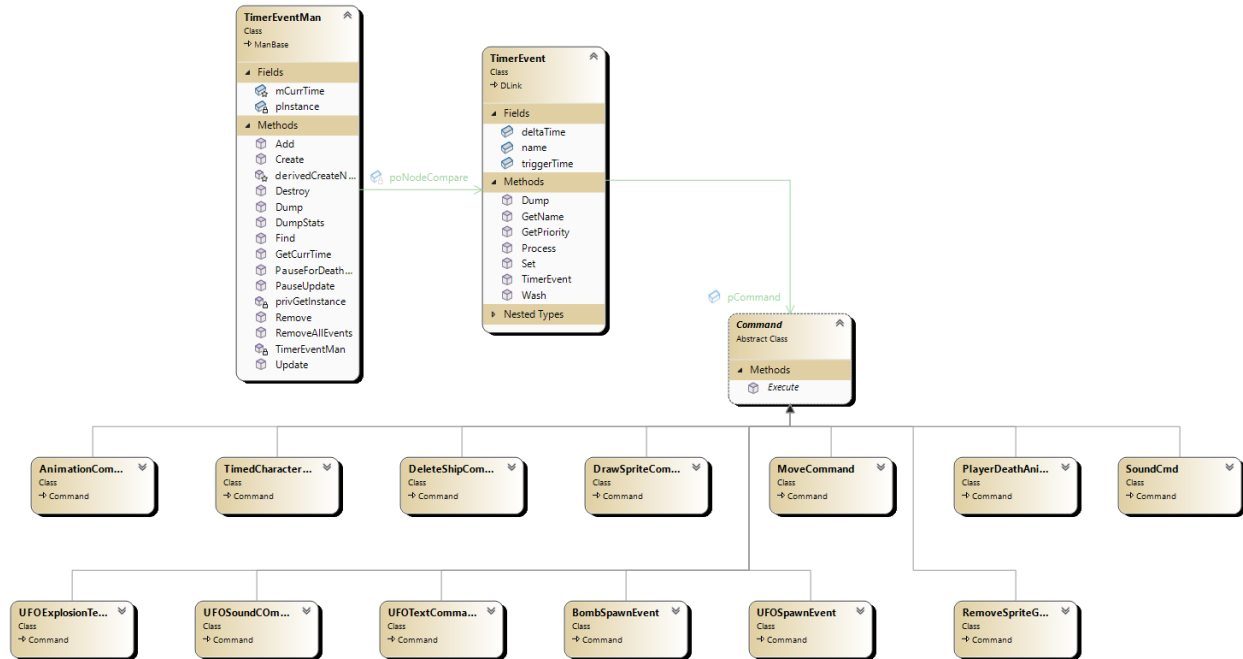
In our **Space Invaders** project, we needed a way to **manage and execute actions** such as **alien sprite animations, alien movements, sound playing**, and other **time-dependent events** efficiently. Some of these actions had to be executed at specific intervals, while others needed to reoccur as part of a sequence. For instance, **alien movement** and **sprite changes** needed to happen at regular intervals.

We wanted a system that could **decouple** the event handling logic from the actual execution of these actions. Additionally, we needed a way to handle **timing** so that actions like **alien movement, bomb spawning, and sound effects** would occur based on the **game timer** without hard-coding the behavior for each action.

Solution:

To solve this challenge, we applied the **Command Pattern** combined with a **TimerEventManager** to handle the timing of events. This solution allowed us to **encapsulate** each action (like **alien movement, sprite animation, and sound playing**) as a **command object** and have them executed at **specific times** as determined by the **TimerEventManager**.

The **TimerEventManager** was responsible for managing and scheduling all **time-dependent events**. It would trigger the appropriate commands (such as **alien movement** or **sound playing**) based on the timing set for each event.



Pattern description

The **Command Pattern** is a behavioral design pattern where an **object** (the **command**) encapsulates a **request** as an object, thereby allowing for **parameterization** of clients with queues, requests, and operations. It consists of the following components:

1. Command Interface (Abstract Command):

- Defines a common interface for all concrete commands. Typically, this interface includes an **execute()** method.

2. Concrete Command:

- These are specific implementations of the **Command** interface. Each concrete command is responsible for performing a particular action, such as moving an alien, triggering sprite animation, or playing a sound.

3. Invoker:

- The **Invoker** is responsible for calling the **execute()** method on the **command** object. In our case, the **TimerEventManager** acted as the **Invoker** by managing and triggering the execution of the commands based on the timing.

4. Receiver:

- The **Receiver** is the object that actually performs the action when a command is executed. For example, the **Alien** object or **SoundManager** object may serve as receivers.

Key Object-Oriented Mechanics

The **Command Pattern** uses several key object-oriented principles to provide flexibility and decoupling:

1. Encapsulation:

- The **Command Pattern** encapsulates all details of an action (such as **alien movement**, **sprite animation**, or **sound playing**) in a command object. The **Invoker** (in this case, **TimerEventManager**) simply calls the **execute()** method without needing to know how the action is performed.

2. Decoupling:

- The **Invoker** and the **Receiver** are decoupled. The **Invoker** (**TimerEventManager**) does not need to know which specific object it is invoking; it simply knows how to execute the **command** associated with a particular event.

3. Flexibility & Extensibility:

- New commands can easily be added without modifying the existing system. If we need to introduce a new behavior (such as **enemy bomb spawning**), we can create a new **command** to handle that behavior without touching the existing codebase.

How it is used in Space Invaders....

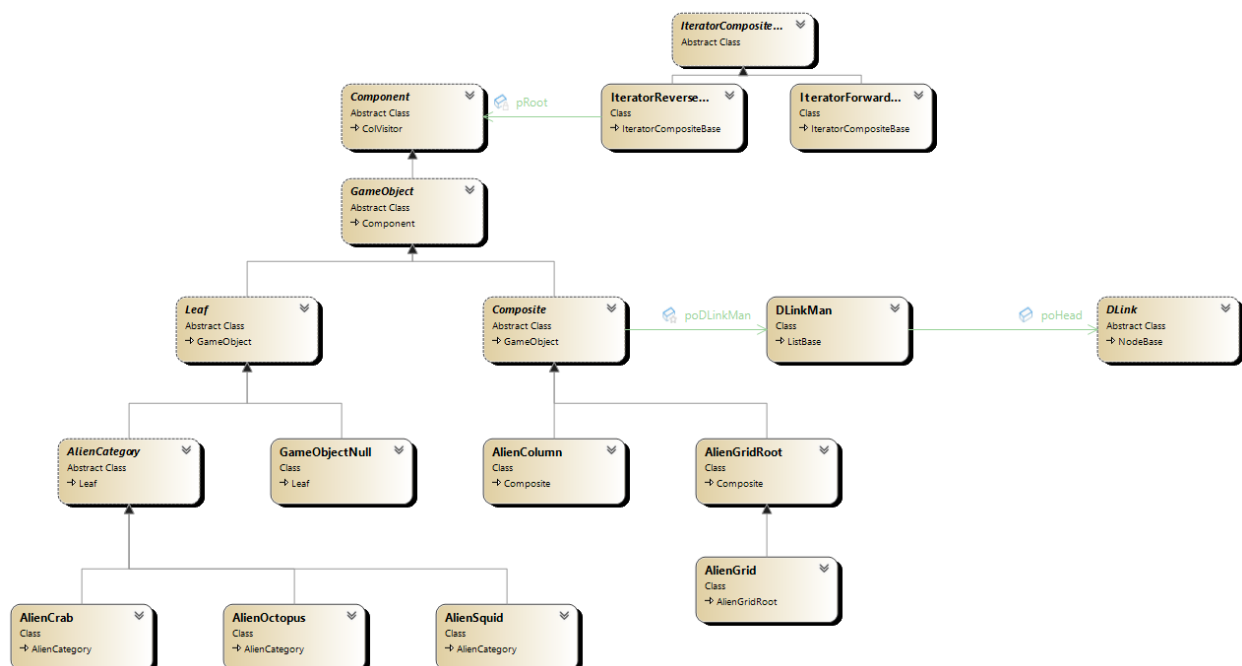
In our **Space Invaders** project, we used the **Command Pattern** to manage actions like **alien movement**, **sprite animation**, and **sound playing** while integrating **TimerEventManager** to handle the timing of these events.

The **TimerEventManager** was responsible for scheduling and triggering timed events. It handled all game events that required specific timing, such as **alien movement**, **sprite animation updates**, **bomb spawning**, and **sound effects**.

- The **TimerEventManager** would register **TimerEvent** objects, each of which contained a specific **command** (e.g., **alien movement**, **sprite animation**) and a **time interval**. When the **timer** reached the specified time, the corresponding command would be executed. We need a way for the **TimerEventManager** to hold **TimerEvents** in their correct order so we implemented **addWithPriority**. This allowed us to add **timerEvents** in their correct order so that we could ensure that we were not wasting time checking each node to see if they should fire. If the current time was lower than the execution time, we could safely have an early exit.
- **AnimationCommand** was used to animate the **AlienSprites** as they moved. The **Command** held a single linked list of the animation states that the **Alien** would go through. Each **alien** had 2 images that they would flip between
 - When the command was executed, it would flip the image then add itself back into the timer.

- The timer was calculated to scale linearly with the amount of aliens left on the grid.
- **MoveCommand** was used to move the entire alien grid left or right.
 - The speed and distance that the aliens moved were scaled linearly with the amount of aliens left on the grid.
- **RemoveSpriteGameCommand** was used to remove the Sprite that we created when a collision occurred. The observer was responsible for adding this event to remove the sprite they created.
- **PlayerDeathAnimation** is the first command called when the observer notifies a collision between the player ship and a bomb. The observer pauses all events except for this animation effectively pausing the game. The observer also has logic to stop the bombs from falling but not the UFO. Stopping the UFO was originally included but from the demo I saw the UFO continues moving even during the death animation.
 - The ship animation cycles between two images of the destroyed ship similar to how to aliens are animated. The ship animates for a designated amount of times by readding itself to the TimerEventManager. After it animates the designated amount of times it stops adding itself back.
- **DeleteShipCommand** is the Second Command Called in the player death observer
 - After a 4 second delay, The Ship is respawned at the left near the spare lives.
 - The ship game object holds the current lives that the player has.
 - The lives are decremented and enters a switch case removing the sprites that represent the lives depending on how many lives are left.
 - When there are 0 lives left the scene is switched to the game over screen.
 - If the player still has lives left they regain control of their ship and are allowed to shoot again by switching the move state to both directions and missile state to ready
 - The shipSprite is switched back to the alive ship and flips the ship is dying flag allowing the ship to be killed once again
 - Here we again loop through all of the bombs enabling their movement unpausing the game
- **DrawSpriteCommand** is used in the SceneIntro. It simply takes a spriteGame name and draws it at the specified x and y location. This is used to show the value of the aliens on the intro screen
- **UFOExplosionTextCommand** is called during the RemoveUFO observer.
 - This command is similar to the update score Observer. Instead of adding the value of the UFO we simply use this command to print the value onto the screen.
 - This is done by once again checking the ship shot count and entering a switch statement.
 - Depending on the value we draw the Font on the screen with its x and y adjusted for different value length.
 - This command then calls the following command
- **UFOTextCommand** is the cleanup command called after the explosion.
 - After a delay it moves the score off screen

- **UFOSoundCommand** this command simply checks if the UFO is currently on the screen and plays the UFO sound.
 - As long as the UFO is on the screen it continuously adds itself back into the EventMan.
- **BombSpawnEvent** is a command which randomly drops a bomb from an alien
 - Depending on the current level of the game more bomb spawn events are added to the TimerEventMan.
 - First the bomb checks a bool that we set to signal whether or not the command is coming from a UFO or not.
 - If it is a UFO then we iterate through the UFO root to find the UFO and randomly 1 of 3 bombs from the bottom of the UFO
 - If it is an Alien then we iterate through the AlienGrid choosing a random Column from which we will drop the bomb
 - From there we randomly choose 1 of 3 bombs to drop.
 - The bomb is created from the lowest alien on the column
 - The bomb is then scaled up and attached to the bombRoot
 - If the object dropping a bomb is a UFO then the timer is not readded
 - If the object is an Alien then another bomb is dropped after 0-5 seconds
- **UFOSpawnEvent** is a command that creates a UFO every 25 seconds.
 - It randomly chooses to spawn going left to right or right to left.
 -



Pattern description

The **Composite Pattern** is a structural design pattern used to compose objects into **tree-like structures** to represent part-whole hierarchies. It allows us to treat both **individual objects** and **composite objects** uniformly. The pattern typically involves the following components:

1. Component (Abstract Component):

- This is an abstract class or interface that defines common behavior for both individual objects and composite objects. It can declare methods that will be implemented by both **leaf objects** (individual elements) and **composite objects** (groups of elements).

2. Leaf (Leaf Node):

- The **Leaf** represents **individual objects** that do not have children. These are the basic building blocks of the hierarchy. For example, in our game, each **Alien** or **ShieldBrick** would be a leaf node in the hierarchy.

3. Composite (Composite Node):

- The **Composite** represents **groups of objects** that can contain other **Leaf** or **Composite** objects. It is responsible for managing and interacting with its children. For example, the **AlienColumn** or **ShieldColumn** would be a composite, as they contain multiple **Alien** or **ShieldBrick** objects, respectively.

Key Object-Oriented Mechanics

The **Composite Pattern** utilizes several object-oriented principles to structure complex hierarchies in a flexible and extensible manner:

1. Polymorphism:

- The **Composite Pattern** allows both **Leaf** and **Composite** objects to implement a common interface, which enables the client to treat both types uniformly. This reduces the complexity of dealing with different object types in the hierarchy.

2. Encapsulation:

- The **Composite** class encapsulates the logic of managing its child objects, ensuring that the client code does not need to be concerned with the structure or organization of the hierarchy. The client simply interacts with the common interface.

3. Flexibility and Extensibility:

- The **Composite Pattern** allows for easy expansion of the hierarchy. New **Leaf** or **Composite** objects can be added without altering the existing codebase. This promotes a **modular** and **extensible** design.

How it is used in Space Invaders....

In our **Space Invaders** project, the **Composite Pattern** was instrumental in organizing and managing the game world's hierarchy. Below are a few examples of how we applied the **Composite Pattern** to manage **GameObjects**:

1. AlienGrid:

- The **AlienGrid** is a composite object that represents the entire grid of aliens. It consists of multiple **AlienColumns**, each of which contains multiple **Aliens**. By using the **Composite Pattern**, we could treat the entire grid of aliens as a single entity when performing actions like **collision detection** or **movement**. When necessary we were able to traverse the composite to detect finer collisions such as single Aliens vs Missiles

2. AlienColumn:

- The **AlienColumn** is a composite object that holds multiple **Aliens**.

3. ShieldRoot and ShieldColumn (Composite):

- Similar to the alien objects, the **ShieldRoot** holds a composite of **ShieldColumns**, which contain **ShieldBricks**.

4. BombRoot:

- The **BombRoot** is a composite of objects that holds multiple **Bombs**.

5. BumperRoot:

- The **BumperRoot** is a composite of objects that hold BumperCategorys which hold Left and Right **Bumpers**

6. MissileGroup:

- The MissileGroup holds Missile objects

7. ShipRoot holds Ships

8.

OBJECT POOL Pattern

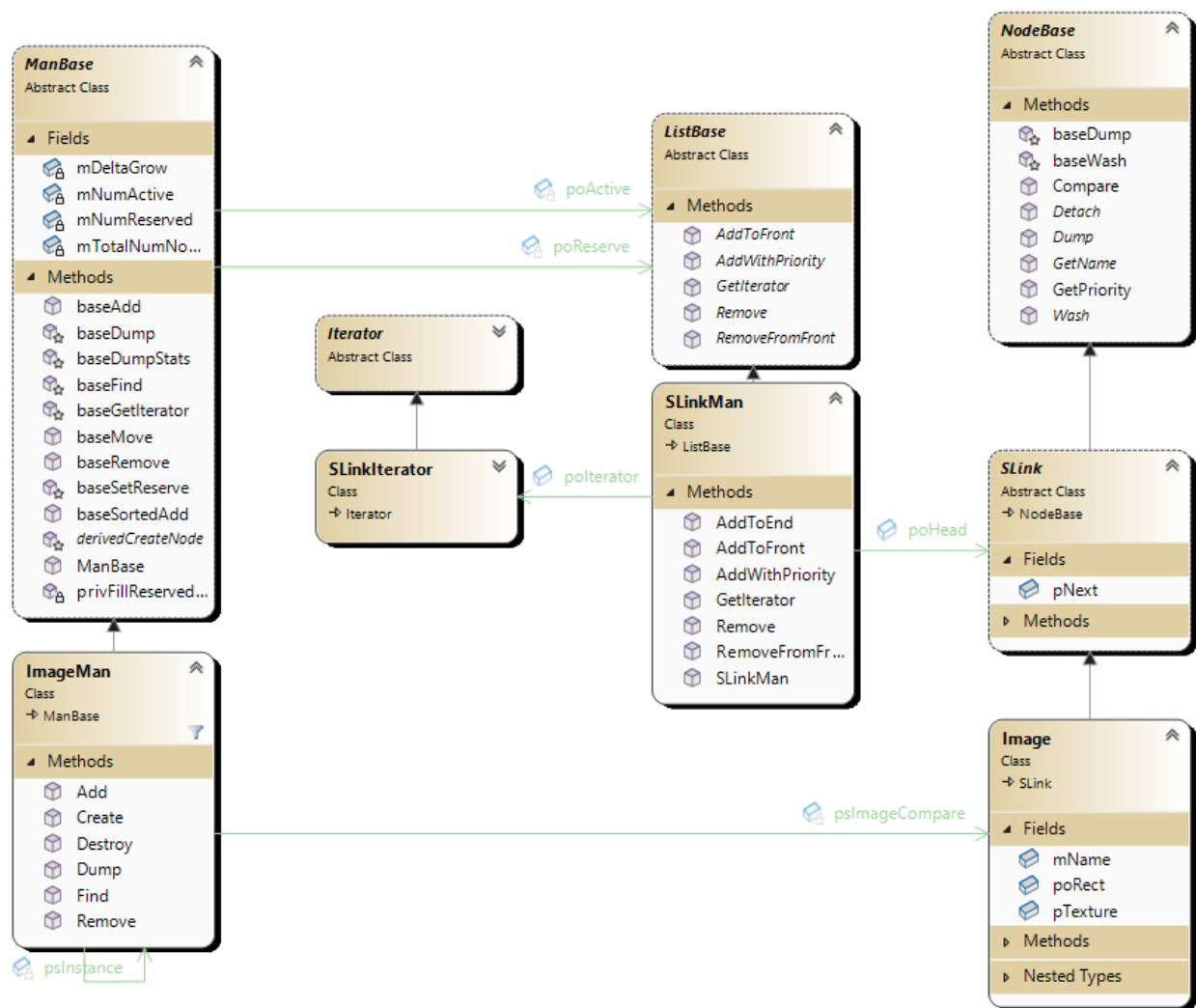
Challenge: Efficient Memory Management for Frequently Created and Deleted Nodes

Problem:

In our **Space Invaders** project, in all of our Singletons we utilized either a single or double linked list. We needed a way to optimize the creation and potential reuse of these nodes.

Solution:

To solve this problem, we implemented the **Object Pool Pattern**. This pattern allowed us to reuse objects **instead of constantly creating and destroying them**. When an object was no longer needed, it was returned to the pool rather than being deleted. Later, when a new object was required, it was retrieved from the pool instead of creating a new instance.



Pattern description

1. The **Object Pool Pattern** is a **creational design pattern** that manages a set of reusable objects. Instead of creating and destroying objects frequently, we maintain a pool of objects and recycle them when needed.
2. **Component (Abstract Component):**
 - This is an abstract class or interface that defines common behavior for both individual objects and composite objects. It can declare methods that will be implemented by both **leaf objects** (individual elements) and **composite objects** (groups of elements).
3. **Leaf (Leaf Node):**
 - The **Leaf** represents **individual objects** that do not have children. These are the basic building blocks of the hierarchy. For example, in our game, each **Alien** or **ShieldBrick** would be a leaf node in the hierarchy.
4. **Composite (Composite Node):**
 - The **Composite** represents **groups of objects** that can contain other **Leaf** or **Composite** objects. It is responsible for managing and interacting with its children. For example, the **AlienColumn** or **ShieldColumn** would be a composite, as they contain multiple **Alien** or **ShieldBrick** objects, respectively.

Key Object-Oriented Mechanics

Object Pools allow for reusable objects.

How it is used in Space Invaders....

Almost every singleton we used included object pooling for managing the nodes on the linked list.

TEMPLATE Pattern

Challenge: Standardizing Game Logic While Allowing Customization

Problem:

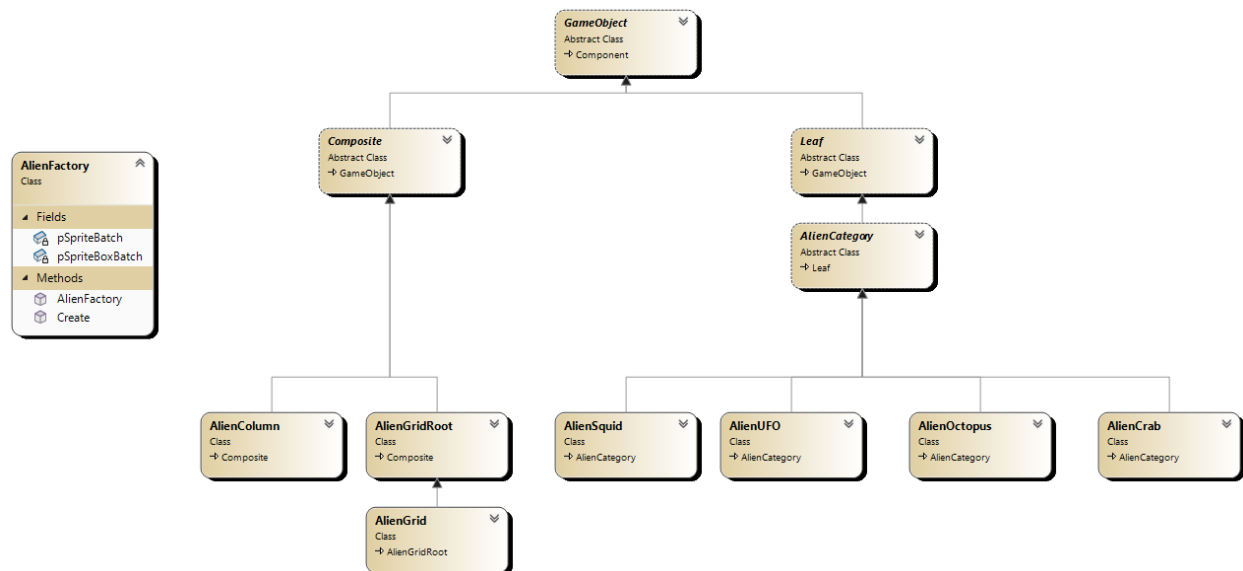
In our **Space Invaders** project, we had multiple game objects and behaviors that followed a similar structure but required slight variations. For example, different Aliens, different commands, and different observers all shared a common base but had unique implementations for certain parts.

Writing separate implementations for each variation would lead to **code duplication** and make maintenance difficult. We needed a way to define a **standard algorithm structure** while allowing subclasses to modify specific steps of the process.

Solution:

To address this issue, we applied the **Template Pattern**. This pattern allows us to **define the skeleton of an object in a base class** while letting subclasses **override specific steps** without altering the overall structure.

By using the **Template Pattern**, we were able to create **base classes** for game mechanics, ensuring that **common logic** remained consistent across different object types while allowing for **customization** where needed.



Pattern description

The **Template Pattern** is a behavioral design pattern that defines the **structure of an object in a base class** while allowing subclasses to override certain steps without changing the overall object.

It consists of the following components:

1. **Abstract Base Class (Template Class)**

- Defines the **skeleton** of an operation, consisting of multiple steps.
- Implements steps that are common across all subclasses.
- Includes **abstract or virtual methods** for steps that subclasses should customize.

2. **Concrete Subclasses**

- Implement the steps that need to be customized while inheriting the rest of the structure from the base class.

Key Object-Oriented Mechanics

Encapsulation:

- The **core algorithm structure** is defined in one place (the base class), keeping the logic centralized and easier to maintain.

Polymorphism:

- Subclasses provide their own implementations of certain steps, ensuring flexibility while maintaining a consistent structure.

Code Reusability:

- Common behavior is defined once in the **template method**, eliminating code duplication across subclasses.

How it is used in Space Invaders....

Almost everything used in our Space Invaders project utilized some Form of the Template Pattern.

Every Manager Inherited from ManBase which included a ListBase which both SLinks and DLinks inherit from. Composites and Leaves inherited from GameObjects and so on.